⚛️ REACTRON

> Jeu console en C : manipule la matrice du réacteur quantique pour le stabiliser avant la surcharge.

REACTRON est un jeu de type *match-3* en console Windows, développé en C dans le cadre du projet d'informatique de l'**ECE Paris** (ING1). Une grille de 12 × 20 éléments colorés représente le cœur d'un réacteur : tu échanges des éléments voisins pour créer des alignements, les absorber et récolter l'énergie demandée avant la fin du temps ou du nombre de manipulations.

🎮 Principe du jeu

- La grille contient 5 types d'éléments : **R** (rouge), **B** (bleu), **G** (vert), **Y** (jaune), **V** (violet).
- Un alignement de **3 ou 4** éléments identiques (horizontal ou vertical) est absorbé et fait gagner de l'énergie à la couleur correspondante.
- Un alignement de **5** absorbe **tous** les éléments de cette couleur sur la grille.
- Après chaque absorption, la gravité fait tomber les éléments et la grille se régénère (en évitant les alignements automatiques).
- Chaque niveau impose des **objectifs d'énergie**, un **nombre maximal de manipulations** et une **limite de temps**.

Niveaux

Niveau	Objectifs d'énergie	Manipulations max	Temps
1	R = 10, B = 10	15	120 s
2	R = 20, B = 20, G = 20	12	75 s
3	R, B, G, Y, V = 25 chacun	30	60 s

Défaite et surcharge

Si tu échoues (plus de manipulations ou temps écoulé), tu peux **recommencer** ou **abandonner**. Chaque nouvelle tentative augmente le **niveau de surcharge** du réacteur. Au-delà de **5**, le réacteur explose : game over définitif.

🕹️ Commandes

Touche	Action
`Z`	Déplacer le curseur vers le haut
`W`	Déplacer le curseur vers le bas
`Q`	Déplacer le curseur vers la gauche
`S`	Déplacer le curseur vers la droite
`Espace`	Sélectionner l'élément sous le curseur
`Z` / `W` / `Q` / `S` (après `Espace`)	Échanger l'élément sélectionné avec son voisin dans cette direction
`Espace`, `F`, `F`	Abandonner la partie

🛠️ Installation et compilation

Prérequis

- **Windows** (le jeu utilise `windows.h` et `conio.h`)
- Un compilateur C : MinGW / GCC ou Code::Blocks

Compilation avec GCC

```
git clone https://github.com/sossou20/REATRON-video-game.git
cd REATRON-video-game
gcc main.c fonctions.c affichage.c -o reactron.exe
reactron.exe
```

Avec Code::Blocks

Crée un projet console C, ajoute les 5 fichiers sources (`main.c`, `fonctions.c`, `fonctions.h`, `affichage.c`, `affichage.h`), puis lance **Build and run**.

> 💡 Agrandis la fenêtre de la console : la grille et le tableau de bord doivent tenir à l'écran.

📁 Structure du projet

```
REATRON-video-game/
├── main.c          # Boucle principale, niveaux, gestion des touches
├── fonctions.c     # Logique du jeu : génération, alignements, gravité, score, r
├── fonctions.h     # Prototypes et constantes
├── affichage.c     # Gestion de la console Windows (couleurs, curseur, effacemen
├── affichage.h     # Prototypes et palette de couleurs
└── README.md
```

Fonctions principales

Module	Fonctions
Génération	`matrice()` (grille initiale sans alignement), `gnr()` (régénération)
Détection	`verifierAlignement()`, `detect()`
Absorption	`suppr()`, `grav()`
Score	`score()`, `augscore()`, `affscore()`
Règles	`chargerObjectifsNiveau()`, `verifierVictoire()`, `verifierEchec()`
Affichage	`titre()`, `afficher_reacteur()`, `afficherObjet()`, `deplacerChasseur()`, `gotoxy()`, `text_color()`

👥 Équipe

Membre	Rôle
Finoana Mahery **RAKOTOMANANA**	Génération de la matrice
Thomas Javan **DJOUBISSIE**	Absorption des éléments
Emmanuel **SOSSOU**	Gestion de l'affichage
Karl Orphé Elom **KPADOUNOU**	Conditions de victoire, échec et surcharge

---

📚 Compétences mises en œuvre

Tableaux bidimensionnels · Allocation dynamique · Programmation modulaire · Bibliothèque `windows.h` · Git / GitHub · Doxygen

---

🏫 Contexte

Projet réalisé à l'**ECE Paris – École d'ingénieurs** (ING1, 2026).